

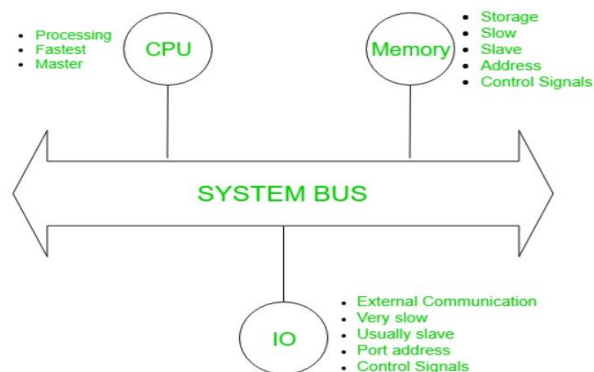
I/O organization

- **Introduction to system buses**
- **Input/output interface**
- **Interrupt and types of interrupts**
- **Serial vs parallel transmission**
- **Synchronous data transfer**
- **Asynchronous data transfer methods**
- **Strobe control**
- **Handshaking**
- **Mode of data transfer**
- **Programmed I/O**
- **DMA**
- **DMA controller**
- **DMA transfer**
- **I/O processor**

Introduction to system buses

The electrically conducting path along which data is transmitted inside any digital electronic device. A Computer bus consists of a set of parallel conductors, which may be conventional wires, copper tracks on a PRINTED CIRCUIT BOARD, or microscopic aluminum trails on the surface of a silicon chip. Each wire carries just one bit, so the number of wires determines the most significant data WORD the bus can transmit: a bus with eight wires can carry only 8-bit data words and hence defines the device as an 8-bit device.

- The bus is a communication channel.
- The characteristic of the bus is shared transmission media.
- The limitation of a bus is only one transmission at a time.
- A bus used to communicate between the major components of a computer is called a **System bus**.



System bus contains 3 categories of lines used to provide the communication between the CPU, memory and IO named as:

1. Address Lines:

- Used to carry the address to memory and IO.
- Unidirectional.
- Based on the width of an address bus we can determine the capacity of a main memory

2. Data Lines:

- Used to carry the binary data between the CPU, memory and IO.

- Bidirectional.
- Based on the width of a data bus we can determine the word length of a CPU.
- Based on the word length we can determine the performance of a CPU.

3. Control Lines:

- Used to carry the control signals and timing signals
- Control signals indicate the type of operation.
- Timing Signals are used to synchronize the memory and IO operations with a CPU clock.
- Typical Control Lines may include Memory Read/Write, IO Read/Write, Bus Request/Grant, etc.

Input/Output Subsystem

The I/O subsystem of a computer provides an efficient mode of communication between the central system and the outside environment. It handles all the input-output operations of the computer system.

Peripheral Devices

Input or output devices that are connected to computer are called **peripheral devices**. These devices are designed to read information into or out of the memory unit upon command from the CPU and are considered to be the part of computer system. These devices are also called **peripherals**.

For example: *Keyboards, display units and printers* are common peripheral devices.

There are three types of peripherals:

1. **Input peripherals** : Allows user input, from the outside world to the computer. Example: Keyboard, Mouse etc.
2. **Output peripherals**: Allows information output, from the computer to the outside world. Example: Printer, Monitor etc
3. **Input-Output peripherals**: Allows both input(from outside world to computer) as well as, output(from computer to the outside world). Example: Touch screen etc.

Interfaces

Interface is a shared boundary between two separate components of the computer system which can be used to attach two or more components to the system for communication purposes.

There are two types of interface:

1. CPU Interface
2. I/O Interface

Input output interface

Input output interface is used as a method which helps in transferring of information between the internal storage devices i.e. memory and the external peripheral device . A peripheral device is that which provide input and output for the computer, it is also called Input-Output devices.

Functions of Input-Output Interface:

1. It is used to synchronize the operating speed of CPU with respect to input-output devices.

2. It selects the input-output device which is appropriate for the interpretation of the input-output device.
3. It is capable of providing signals like control and timing signals.
4. In this data buffering can be possible through data bus.
5. There are various error detectors.
6. It converts serial data into parallel data and vice-versa.
7. It also convert digital data into analog signal and vice-versa.

Mode of data transfer

Programmed I/O :

In this mode the data transfer is initiated by the instructions written in a computer program. An input instruction is required to store the data from the device to the CPU and a store instruction is required to transfer the data from the CPU to the device. Data transfer through this mode requires constant monitoring of the peripheral device by the CPU and also monitor the possibility of new transfer once the transfer has been initiated. Thus CPU stays in a loop until the I/O device indicates that it is ready for data transfer. Thus programmed I/O is a time consuming process that keeps the processor busy needlessly and leads to wastage of the CPU cycles.

This can be overcome by the use of an interrupt facility. This forms the basis for the Interrupt Initiated I/O.

Interrupt Initiated I/O :

This mode uses an interrupt facility and special commands to inform the interface to issue the interrupt command when data becomes available and interface is ready for the data transfer. In the meantime CPU keeps on executing other tasks and need not check for the flag. When the flag is set, the interface is informed and an interrupt is initiated. This interrupt causes the CPU to deviate from what it is doing

to respond to the I/O transfer. The CPU responds to the signal by storing the return address from the program counter (PC) into the memory stack and then branches to service that processes the I/O request. After the transfer is complete, CPU returns to the previous task it was executing. The branch address of the service can be chosen in two ways known as vectored and non-vectored interrupt. In vectored interrupt, the source that interrupts, supplies the branch information to the CPU while in case of non-vectored interrupt the branch address is assigned to a fixed location in memory.

Difference between Programmed and Interrupt Initiated I/O :

Programmed I/O	Interrupt Initiated I/O
Data transfer is initiated by the means of instructions stored in the computer program. Whenever there is a request for I/O transfer the instructions are executed from the program.	The I/O transfer is initiated by the interrupt command issued to the CPU.
The CPU stays in the loop to know if the device is ready for transfer and has to continuously monitor the peripheral device.	There is no need for the CPU to stay in the loop as the interrupt command interrupts the CPU when the device is ready for data transfer.
This leads to the wastage of CPU cycles as CPU remains busy needlessly and thus the efficiency of system gets reduced.	The CPU cycles are not wasted as CPU continues with other work during this time and hence this method is more efficient.
CPU cannot do any work until the transfer is complete as it has to stay in the loop to continuously monitor the peripheral device.	CPU can do any other work until it is interrupted by the command indicating the readiness of device for data transfer

Programmed I/O	Interrupt Initiated I/O
Its module is treated as a slow module.	Its module is faster than programmed I/O module.

It is quite easy to program and understand.	It can be tricky and complicated to understand if one uses low level language.
The performance of the system is severely degraded.	The performance of the system is enhanced to some extent.

DMA CONTROLLER

DMA or Direct Memory Access Controller is an external device that controls the transfer of data between I/O device and memory without the involvement of the processor. It holds the ability to directly access the main memory for read or write operation.

DMA CONTROLLER uses hardware for accessing the memory, that hardware is called a DMA Controller. It has the work of transferring the data between Input Output devices and main memory with very less interaction with the processor. The direct Memory Access Controller is a control unit, which has the work of transferring data.

DMA Controller Diagram in Computer Architecture

DMA Controller is a type of control unit that works as an interface for the data bus and the I/O Devices. As mentioned, DMA Controller has the work of transferring the data without the intervention of the processors, processors can control the

data transfer. DMA Controller also contains an address unit, which generates the address and selects an I/O device for the transfer of data. Here we are showing the block diagram of the DMA Controller.

Types of Direct Memory Access (DMA)

There are four popular types of DMA.

- Single-Ended DMA
- Dual-Ended DMA
- Arbitrated-Ended DMA
- Interleaved DMA

Single-Ended DMA: Single-Ended DMA Controllers operate by reading and writing from a single memory address. They are the simplest DMA.

Dual-Ended DMA: Dual-Ended DMA controllers can read and write from two memory addresses. Dual-ended DMA is more advanced than single-ended DMA.

Arbitrated-Ended DMA: Arbitrated-Ended DMA works by reading and writing to several memory addresses. It is more advanced than Dual-Ended DMA.

Interleaved DMA: Interleaved DMA are those DMA that read from one memory address and write from another memory address.

Modes of Data Transfer in DMA

There are 3 [modes of data transfer](#) in DMA that are described below.

- **Burst Mode:** In Burst Mode, buses are handed over to the CPU by the DMA if the whole data is completely transferred, not before that.
- **Cycle Stealing Mode:** In Cycle Stealing Mode, buses are handed over to the CPU by the DMA after the transfer of each byte. Continuous request for bus control is generated by this Data Transfer Mode. It works more easily for higher-priority tasks.

- **Transparent Mode:** Transparent Mode in DMA does not require any bus in the transfer of the data as it works when the CPU is executing the transaction.

8237 DMA Controller

[8237 DMA Controller](#) is a type of DMA Controller which has a flexible number of channels but generally works on 4 Input-Output channels. In these present channels, the channel has to be given the highest priority to be decided by the Priority Encoder. Each channel in the 8237 DMA Controller has to be programmed separately.

8257 DMA Controller

8257 DMA Controller is a type of DMA Controller, that when a single Intel 8212 I/O device is paired with it, becomes 4 channel DMA Controller. In 8257 DMA Controller, the highest priority channel is acknowledged. It contains two 16-bit registers, one is DMA Address Register and the other one is Terminal Count Register.

Interrupt and types of interrupt

Priority Interrupt

A priority interrupt is a system which decides the priority at which various devices, which generates the interrupt signal at the same time, will be serviced by the CPU. The system has authority to decide which conditions are allowed to interrupt the CPU, while some other interrupt is being serviced. Generally, devices with high speed transfer such as *magnetic disks* are given high priority and slow devices such as *keyboards* are given low priority.

When two or more devices interrupt the computer simultaneously, the computer services the device with the higher priority first.

The interrupt is a signal emitted by hardware or software when a process or an event needs immediate attention. It alerts the processor to a high-priority process requiring interruption of the current working process. In I/O devices one of the bus control lines is dedicated for this purpose and is called the *Interrupt Service Routine (ISR)*.

Types of Interrupts:

Following are some different types of interrupts:

Hardware Interrupts

When the signal for the processor is from an external device or hardware then this interrupts is known as **hardware interrupt**.

Let us consider an example: when we press any key on our keyboard to do some action, then this pressing of the key will generate an interrupt signal for the processor to perform certain action. Such an interrupt can be of two types:

- **Maskable Interrupt**

The hardware interrupts which can be delayed when a much high priority interrupt has occurred at the same time.

- **Non Maskable Interrupt**

The hardware interrupts which cannot be delayed and should be processed by the processor immediately.

Software Interrupts

The interrupt that is caused by any internal system of the computer system is known as a **software interrupt**. It can also be of two types:

- **Normal Interrupt**

The interrupts that are caused by software instructions are called **normal software interrupts**.

- **Exception**

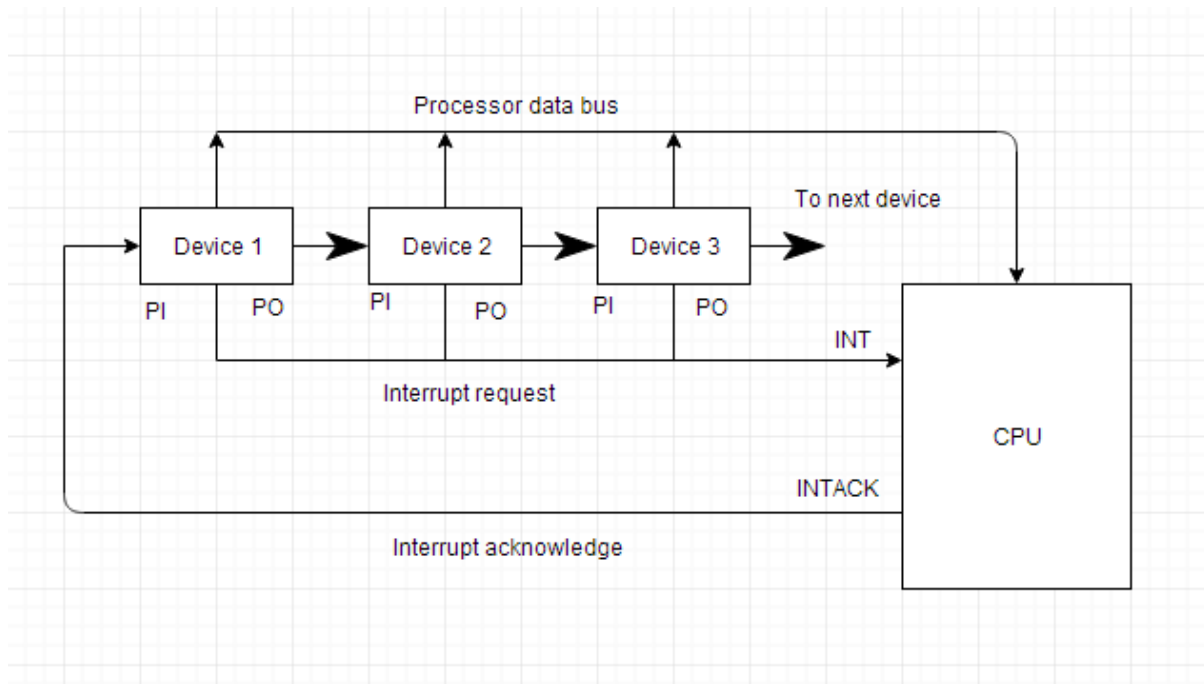
Unplanned interrupts which are produced during the execution of some program are called **exceptions**, such as division by zero.

Daisy Chaining Priority

This way of deciding the interrupt priority consists of serial connection of all the devices which generates an interrupt signal. The device with the highest priority is placed at the first position followed by lower priority devices and the device which has lowest priority among all is placed at the last in the chain.

In daisy chaining system all the devices are connected in a serial form. The interrupt line request is common to all devices. If any device has interrupt signal in low level state then interrupt line goes to low level state and enables the interrupt input in the CPU. When there is no interrupt the interrupt line stays in high level state. The CPU respond to the interrupt by enabling the interrupt acknowledge line. This signal is received by the device 1 at its PI input. The acknowledge signal passes to next device through PO output only if device 1 is not requesting an interrupt.

The following figure shows the block diagram for daisy chaining priority system.



Synchronous data transfer

In Synchronous Data Transfer, the sending and receiving units are enabled with the same clock signal. It is possible between two units when each of them knows the behaviour of the other. The master performs a sequence of instructions for data transfer in a predefined order. All these actions are synchronized with the common clock. The master is designed to supply the data at a time when the slave is definitely ready for it.

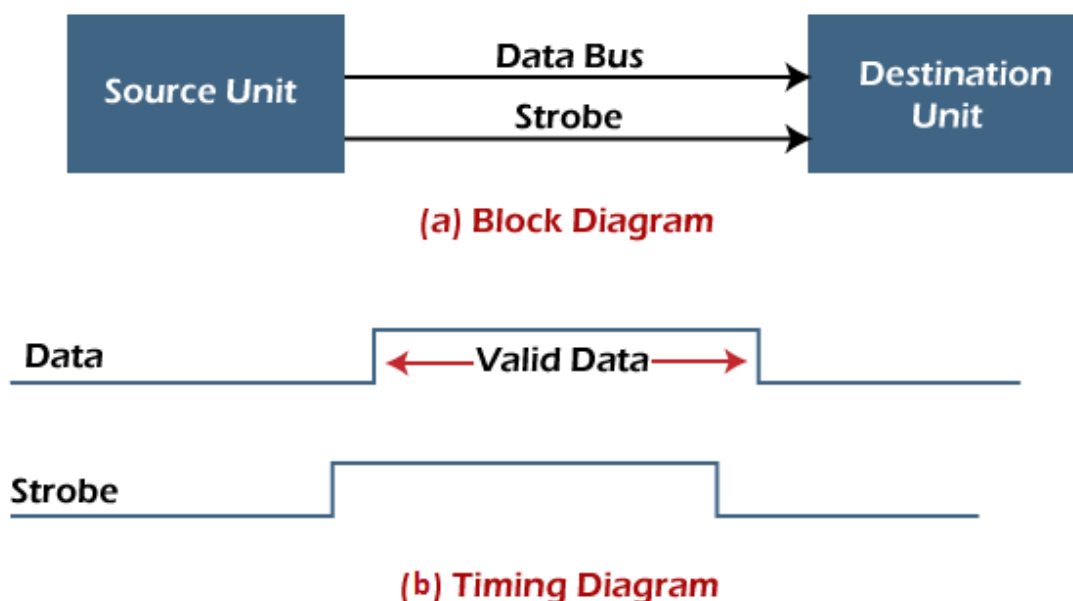
Asynchronous Synchronous Data Transfer methods

The asynchronous data transfer between two independent units requires that control signals be transmitted between the communicating units to indicate when they send the data. Thus, the two methods can achieve the asynchronous way of data transfer.

1. Strobe Control Method

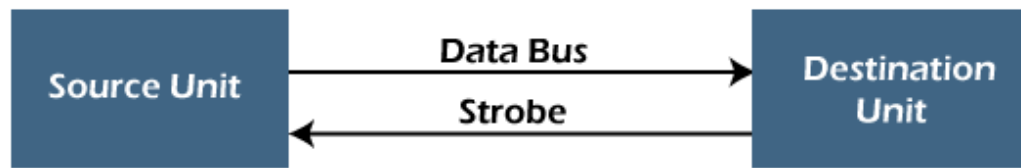
The Strobe Control method of asynchronous data transfer employs a single control line to time each transfer. This control line is also known as a strobe, and it may be achieved either by source or destination, depending on which initiates the transfer.

Source initiated strobe: In the below block diagram, you can see that strobe is initiated by source, and as shown in the timing diagram, the source unit first places the data on the data bus.

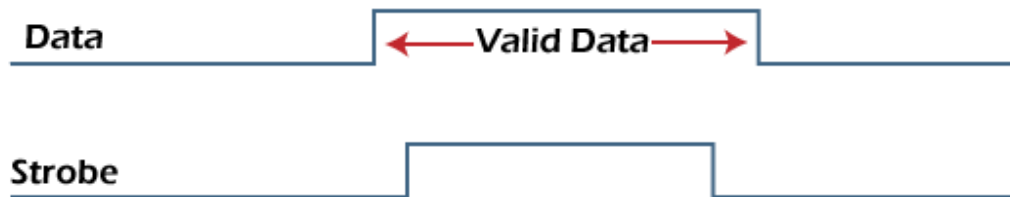


Destination initiated strobe: In the below block diagram, you see that the strobe is initiated by destination, and in the timing diagram, the destination unit first activates the strobe pulse,

informing the source to provide the data.



(a) Block Diagram



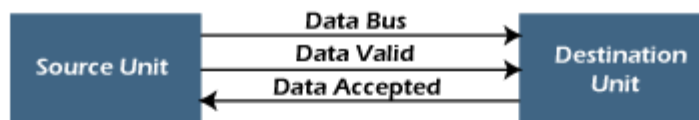
(b) Timing Diagram

Handshaking Method

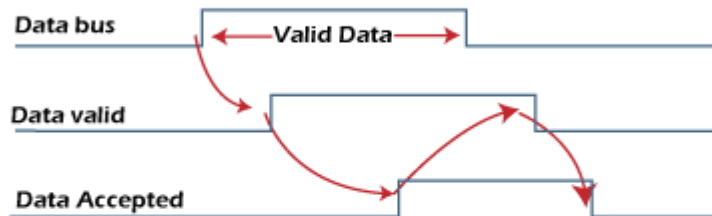
The strobe method has the disadvantage that the source unit that initiates the transfer has no way of knowing whether the destination has received the data that was placed in the bus. Similarly, a destination unit that initiates the transfer has no way of knowing whether the source unit has placed data on the bus.

Source initiated handshaking: In the below block diagram, you can see that two handshaking lines are "**data valid**", which is generated by the source unit, and "**data accepted**", generated by the

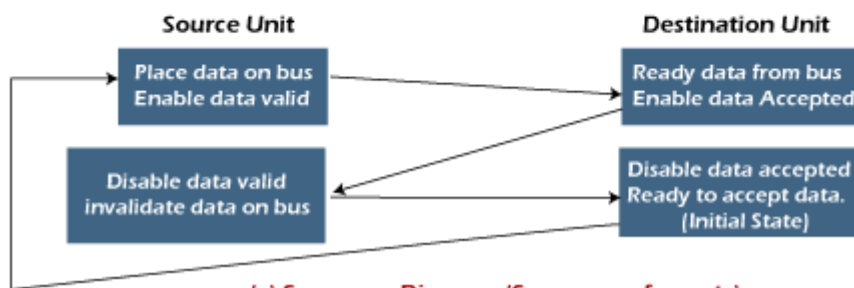
destination unit.



(a) Block Diagram



(b) Timing Diagram



(c) Sequence Diagram (Sequence of events)

Advantages of Asynchronous Data Transfer

Asynchronous Data Transfer in computer organization has the following advantages, such as:

- It is more flexible, and devices can exchange information at their own pace. In addition, individual data characters can complete themselves so that even if one packet is corrupted, its predecessors and successors will not be affected.
- It does not require complex processes by the receiving device. Furthermore, it means that inconsistency in data transfer does not result in a big crisis since the device can keep up with the data

stream. It also makes asynchronous transfers suitable for applications where character data is generated irregularly.

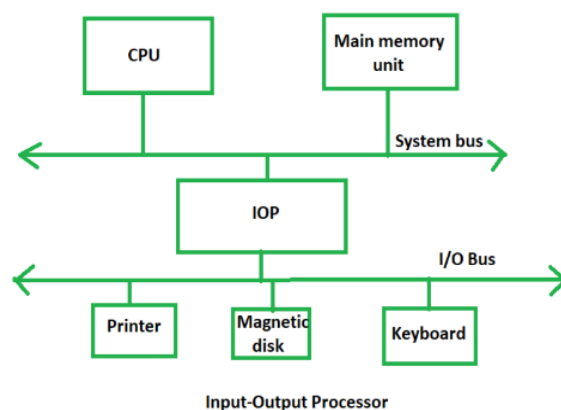
Disadvantages of Asynchronous Data Transfer

There are also some disadvantages of using asynchronous data for transfer in computer organization, such as:

- The success of these transmissions depends on the start bits and their recognition. Unfortunately, this can be easily susceptible to line interference, causing these bits to be corrupted or distorted.
- A large portion of the transmitted data is used to control and identify header bits and thus carries no helpful information related to the transmitted data. This invariably means that more data packets need to be sent.

I/O processor

The Input-Output Processor is a specialized processor which loads and stores data in memory along with the execution of I/O instructions. It acts as an interface between the system and devices. It involves a sequence of events to execute I/O operations and then store the results in memory.



Features of an Input-Output Processor

- **Specialized Hardware:** An IOP is equipped with specialized hardware that is optimized for handling input/output operations. This hardware includes input/output ports, DMA controllers, and interrupt controllers.
- **DMA Capability:** An IOP has the capability to perform Direct Memory Access (DMA) operations. DMA allows data to be transferred directly between peripheral devices and memory without going through the CPU, thereby freeing up the CPU for other tasks.
- **Interrupt Handling:** An IOP can handle interrupts from peripheral devices and manage them independently of the CPU. This allows the CPU to focus on executing application programs while the IOP handles interrupts from peripheral devices.
- **Protocol Handling:** An IOP can handle communication protocols for different types of devices such as [Ethernet](#), USB, and SCSI. This allows the IOP to interface with a wide range of devices without requiring additional software support from the CPU.
- **Buffering:** An IOP can buffer data between the CPU and peripheral devices. This allows the IOP to handle large amounts of data without overloading the CPU or the peripheral devices.
- **Command Processing:** An IOP can process commands from peripheral devices independently of the CPU. This allows the CPU to focus on executing application

programs while the IOP handles peripheral device commands.

- **Parallel Processing:** An IOP can perform input/output operations in parallel with the CPU. This allows the system to handle multiple tasks simultaneously and improve overall system performance.

Applications of I/O Processors

- **Data Acquisition Systems:** I/O processors can be used in data acquisition systems to acquire and process data from various sensors and input devices. The I/O processor can handle high-speed data transfer and perform real-time processing of the acquired data.
- **Industrial Control Systems:** I/O processors can be used in industrial control systems to interface with various control devices and sensors. The I/O processor can provide precise timing and control signals, and can also perform local processing of the input data.
- **Multimedia Applications:** I/O processors can be used in multimedia applications to handle the input and output of multimedia data, such as audio and video. The I/O processor can perform real-time processing of multimedia data, including decoding, encoding, and compression.
- **Network Communication Systems:** I/O processors can be used in network communication systems to handle the input and output of data packets. The I/O processor can perform packet routing, filtering, and processing, and can also perform encryption and decryption of the data.

- **Storage Systems:** I/O processors can be used in storage systems to handle the input and output of data to and from storage devices. The I/O processor can handle high-speed data transfer and perform data caching and prefetching operations.

Advantages of Input-Output Processor

- The I/O devices can directly access the main memory without the intervention of the processor in I/O processor-based systems.
- It is used to address the problems that arise in the Direct memory access method.
- **Reduced Processor Workload:** With an I/O processor, the main processor doesn't have to deal with I/O operations, allowing it to focus on other tasks. This results in more efficient use of the processor's resources and can lead to faster overall system performance.
- **Improved Data Transfer Rates:** Since the I/O processor can access memory directly, data transfers between I/O devices and memory can be faster and more efficient than with other methods.

Disadvantages of Input-Output Processor\

- **Cost:** I/O processors can add significant costs to a system due to the additional hardware and complexity

required. This can be a barrier to adoption, especially for smaller systems.

- **Increased Complexity:** The addition of an I/O processor can increase the overall complexity of a system, making it more difficult to design, build, and maintain. This can also make it harder to diagnose and troubleshoot issues.
- **Limited Performance Gains:** While I/O processors can improve system performance by offloading I/O tasks from the main processor, the gains may not be significant in all cases. In some cases, the additional overhead of the I/O processor may actually slow down the system.
- **Synchronization Issues:** With multiple processors accessing the same memory, synchronization issues can arise, leading to potential data corruption or other errors.
- **Lack of Standardization:** There are many different I/O processor architectures and interfaces available, which can make it difficult to develop standardized software and hardware solutions. This can limit interoperability and make it harder for vendors to develop compatible products.